AI Assisted Coding Assignment- 12.5

RUDROJU RUPA SRI

2303A51918

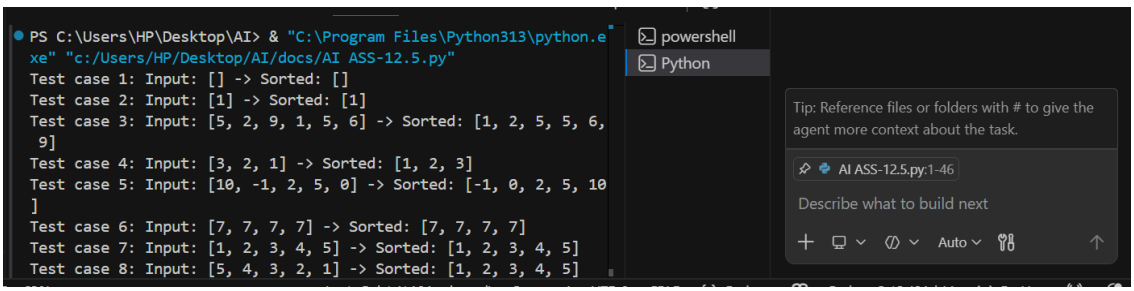
BATCH-30

Lab 12: Algorithms with AI Assistance – Sorting, Searching, and Optimizing Algorithms
Lab Objectives:
• Apply AI-assisted programming to implement and optimize sorting and searching algorithms.
• Compare different algorithms in terms of efficiency and use cases.
• Understand how AI tools can suggest optimized code and complexity improvements.

Task Description #1 (Sorting – Merge Sort Implementation)
• Task: Use AI to generate a Python program that implements the Merge Sort algorithm.
• Instructions:
o Prompt AI to create a function merge_sort(arr) that sorts a list in ascending order.
o Ask AI to include time complexity and space complexity in the function docstring.
o Verify the generated code with test cases.
• Expected Output:
o A functional Python script implementing Merge Sort with proper documentation.

```
#Task-1 (Sorting – Merge Sort Implementation)
#Prompt:Generate a Python program that create a function merge_sort(arr) that
sorts a list in ascending order,by including time complexity and space comple
xity in the function docstring, with test cases.
def merge_sort(arr):
    """
    Sorts a list in ascending order using the merge sort algorithm.
    Time Complexity: O(n log n)
    Space Complexity: O(n)
    Args:
        arr (list): List of elements to be sorted.
    Returns:
        list: Sorted list in ascending order.
```

```
    """
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
# Test cases
if __name__ == "__main__":
    test_cases = [
        [],
        [1],
        [5, 2, 9, 1, 5, 6],
        [3, 2, 1],
        [10, -1, 2, 5, 0],
        [7, 7, 7, 7],
        [1, 2, 3, 4, 5],
        [5, 4, 3, 2, 1]
    ]
    for idx, case in enumerate(test_cases):
        sorted_case = merge_sort(case)
        print(f"Test case {idx + 1}: Input: {case} -> Sorted: {sorted_case}")
```



Task Description #2 (Searching – Binary Search with AI Optimization)

• Task: Use AI to create a binary search function that finds a target element in a sorted list.

• Instructions:

o Prompt AI to create a function binary_search(arr, target) returning the index of the target or -1 if not found.

o Include docstrings explaining best, average, and worst-case complexities.

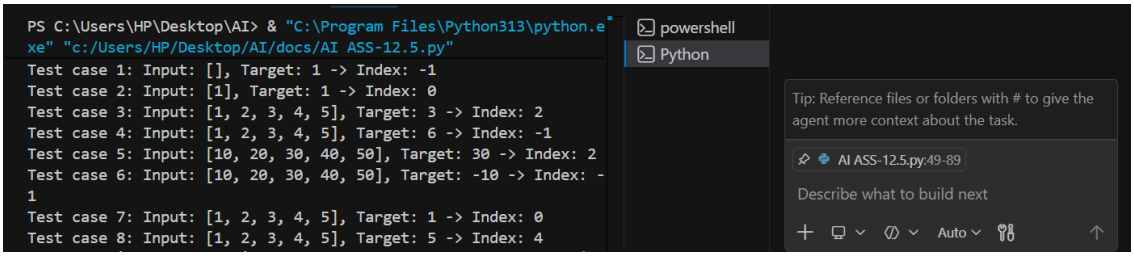
o Test with various inputs.

• Expected Output:

o Python code implementing binary search with AI-generated comments and docstrings.

```
#Task-2 (Searching – Binary Search with AI Optimization)
#Prompt:Create a binary search function binary_search(arr,target) returning t
he index of the target or -1 if not found in a sorted list.By including docst
rings explaining best, average, and worst-case complexities.Test with various
inputs.
def binary_search(arr, target):
    """
    Performs binary search on a sorted list to find the index of the target e
lement.
    Time Complexity:
        Best Case: O(1) - when the target is at the middle of the list.
        Average Case: O(log n) - when the target is found after several itera
tions.
        Worst Case: O(log n) - when the target is not found or is at the end
of the list.
    Space Complexity: O(1) - iterative implementation uses constant space.
    Args:
        arr (list): A sorted list of elements.
        target: The element to search for.
    Returns:
        int: The index of the target element if found, otherwise -1.
    """
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = left + (right - left) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
```

```
            left = mid + 1
        else:
            right = mid - 1
    return -1
# Test cases
if __name__ == "__main__":
    test_cases = [
        ([], 1),
        ([1], 1),
        ([1, 2, 3, 4, 5], 3),
        ([1, 2, 3, 4, 5], 6),
        ([10, 20, 30, 40, 50], 30),
        ([10, 20, 30, 40, 50], -10),
        ([1, 2, 3, 4, 5], 1),
        ([1, 2, 3, 4, 5], 5)
    ]
    for idx, (arr, target) in enumerate(test_cases):
        result = binary_search(arr, target)
        print(f"Test case {idx + 1}: Input: {arr}, Target: {target} -> Index:
{result}")
```



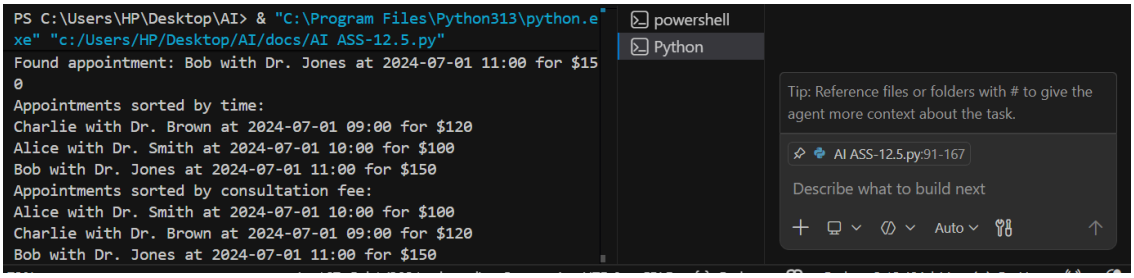
Task Description #3: Smart Healthcare Appointment Scheduling System

A healthcare platform maintains appointment records containing appointment ID, patient name, doctor name, appointment time, and consultation fee. The system needs to:

1. Search appointments using appointment ID.
2. Sort appointments based on time or consultation fee.
   Student Task
   • Use AI to recommend suitable searching and sorting algorithms.
   • Justify the selected algorithms.
   • Implement the algorithms in Python.

```
#Task-3: Smart Healthcare Appointment Scheduling System
#Prompt:A healthcare platform maintains appointment records containing appoin
tment ID, patient name, doctor name, appointment time, and consultation fee.
The system needs to:
#Search appointments using appointment ID.
#Sort appointments based on time or consultation fee.
#Recommend suitable searching and sorting algorithms then justify the selecte
d algorithms.Implement the algorithms in Python.
class Appointment:
    def __init__(self, appointment_id, patient_name, doctor_name, appointment
_time, consultation_fee):
        self.appointment_id = appointment_id
        self.patient_name = patient_name
        self.doctor_name = doctor_name
        self.appointment_time = appointment_time
        self.consultation_fee = consultation_fee
class AppointmentSystem:
    def __init__(self):
        self.appointments = []
    def add_appointment(self, appointment):
        self.appointments.append(appointment)
    def search_appointment_by_id(self, appointment_id):
        for appointment in self.appointments:
            if appointment.appointment_id == appointment_id:
                return appointment
        return None
    # Merge Sort Algorithm to sort appointments by a given key (time or fee)
    def merge_sort(self, appointments, key):
        if len(appointments) > 1:
            mid = len(appointments) // 2
            left_half = appointments[:mid]
            right_half = appointments[mid:]
            # Recursive call on each half
            self.merge_sort(left_half, key)
            self.merge_sort(right_half, key)
            i = j = k = 0
            # Merging the two halves back together
            while i < len(left_half) and j < len(right_half):
                if getattr(left_half[i], key) < getattr(right_half[j], key):
                    appointments[k] = left_half[i]
                    i += 1
                else:
                    appointments[k] = right_half[j]
                    j += 1
```

```
                k += 1
            # Checking for remaining elements in the left or right half
            while i < len(left_half):
                appointments[k] = left_half[i]
                i += 1
                k += 1
            while j < len(right_half):
                appointments[k] = right_half[j]
                j += 1
                k += 1
    def sort_appointments_by_time(self):
        self.merge_sort(self.appointments, 'appointment_time')
    def sort_appointments_by_fee(self):
        self.merge_sort(self.appointments, 'consultation_fee')
# Test cases
if __name__ == "__main__":
    system = AppointmentSystem()
    system.add_appointment(Appointment(1, "Alice", "Dr. Smith", "2024-07-01 1
0:00", 100))
    system.add_appointment(Appointment(2, "Bob", "Dr. Jones", "2024-07-01 11:
00", 150))
    system.add_appointment(Appointment(3, "Charlie", "Dr. Brown", "2024-07-01
09:00", 120))
    # Search for an appointment
    appointment = system.search_appointment_by_id(2)
    if appointment:
        print(f"Found appointment: {appointment.patient_name} with {appointme
nt.doctor_name} at {appointment.appointment_time} for ${appointment.consultat
ion_fee}")
    else:
        print("Appointment not found.")
    # Sort appointments by time
    system.sort_appointments_by_time()
    print("Appointments sorted by time:")
    for appointment in system.appointments:
        print(f"{appointment.patient_name} with {appointment.doctor_name} at
{appointment.appointment_time} for ${appointment.consultation_fee}")
    # Sort appointments by consultation fee
    system.sort_appointments_by_fee()
    print("Appointments sorted by consultation fee:")
    for appointment in system.appointments:
        print(f"{appointment.patient_name} with {appointment.doctor_name} at
{appointment.appointment_time} for ${appointment.consultation_fee}")
```



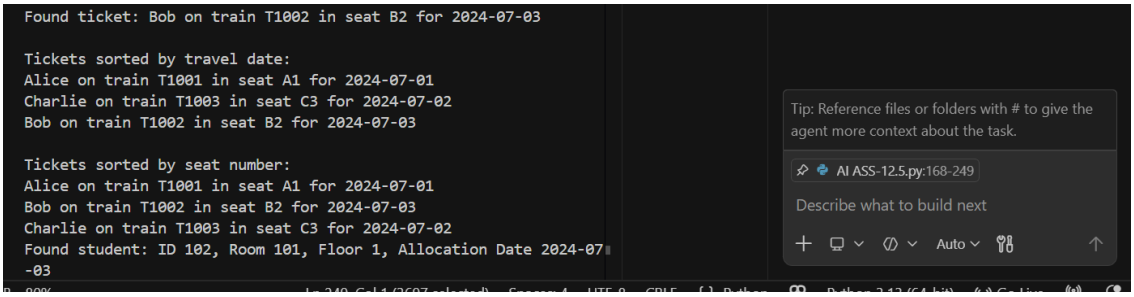
Task Description #4: Railway Ticket Reservation System
Scenario
A railway reservation system stores booking details such as ticket ID, passenger name, train number, seat number, and travel date. The system must:

1. Search tickets using ticket ID.
2. Sort bookings based on travel date or seat number.
   Student Task
   - Identify efficient algorithms using AI assistance.
   - Justify the algorithm choices.
   - Implement searching and sorting in Python.

```
#Task-4: Railway Ticket Reservation System
#Prompt:A railway reservation system stores booking details such as ticket I
D, passenger name, train number, seat number, and travel date. The system mus
t:
#Search tickets using ticket ID.
#Sort bookings based on travel date or seat number.
#Identify efficient algorithms.Justify the algorithm choices.Implement search
ing and sorting in Python.
from asyncio import Task
from datetime import datetime
class Ticket:
    def __init__(self, ticket_id, passenger_name, train_number, seat_number,
travel_date):
        self.ticket_id = ticket_id
        self.passenger_name = passenger_name
        self.train_number = train_number
        self.seat_number = seat_number
        self.travel_date = travel_date
class ReservationSystem:
    def __init__(self):
        self.tickets = []
    def add_ticket(self, ticket):
```

```
        self.tickets.append(ticket)
    # Search tickets by ticket ID
    def search_ticket_by_id(self, ticket_id):
        for ticket in self.tickets:
            if ticket.ticket_id == ticket_id:
                return ticket
        return None
    # Merge Sort algorithm to sort tickets by travel date or seat number
    def merge_sort(self, tickets, key):
        if len(tickets) > 1:
            mid = len(tickets) // 2
            left_half = tickets[:mid]
            right_half = tickets[mid:]
            # Recursively split the list
            self.merge_sort(left_half, key)
            self.merge_sort(right_half, key)
            i = j = k = 0
            # Merge sorted lists
            while i < len(left_half) and j < len(right_half):
                if getattr(left_half[i], key) < getattr(right_half[j], key):
                    tickets[k] = left_half[i]
                    i += 1
                else:
                    tickets[k] = right_half[j]
                    j += 1
                k += 1
            while i < len(left_half):
                tickets[k] = left_half[i]
                i += 1
                k += 1
            while j < len(right_half):
                tickets[k] = right_half[j]
                j += 1
                k += 1
    # Sort tickets by travel date
    def sort_tickets_by_date(self):
        self.merge_sort(self.tickets, 'travel_date')
    # Sort tickets by seat number
    def sort_tickets_by_seat(self):
        self.merge_sort(self.tickets, 'seat_number')
# Test cases
if __name__ == "__main__":
    system = ReservationSystem()
    # Adding sample tickets
    system.add_ticket(Ticket(101, "Alice", "T1001", "A1", datetime(2024, 7,
```

```
1)))
    system.add_ticket(Ticket(102, "Bob", "T1002", "B2", datetime(2024, 7,
3)))
    system.add_ticket(Ticket(103, "Charlie", "T1003", "C3", datetime(2024, 7,
2)))
    # Search for a ticket by ID
    ticket = system.search_ticket_by_id(102)
    if ticket:
        print(f"Found ticket: {ticket.passenger_name} on train {ticket.train_
number} in seat {ticket.seat_number} for {ticket.travel_date.strftime('%Y-%m-
%d')}")
    else:
        print("Ticket not found.")
    # Sort tickets by travel date
    system.sort_tickets_by_date()
    print("\nTickets sorted by travel date:")
    for ticket in system.tickets:
        print(f"{ticket.passenger_name} on train {ticket.train_number} in sea
t {ticket.seat_number} for {ticket.travel_date.strftime('%Y-%m-%d')}")
    # Sort tickets by seat number
    system.sort_tickets_by_seat()
    print("\nTickets sorted by seat number:")
    for ticket in system.tickets:
        print(f"{ticket.passenger_name} on train {ticket.train_number} in sea
t {ticket.seat_number} for {ticket.travel_date.strftime('%Y-%m-%d')}")
```



Task Description #5: Smart Hostel Room Allocation System
A hostel management system stores student room allocation details including student ID, room number, floor, and allocation date. The system needs to:

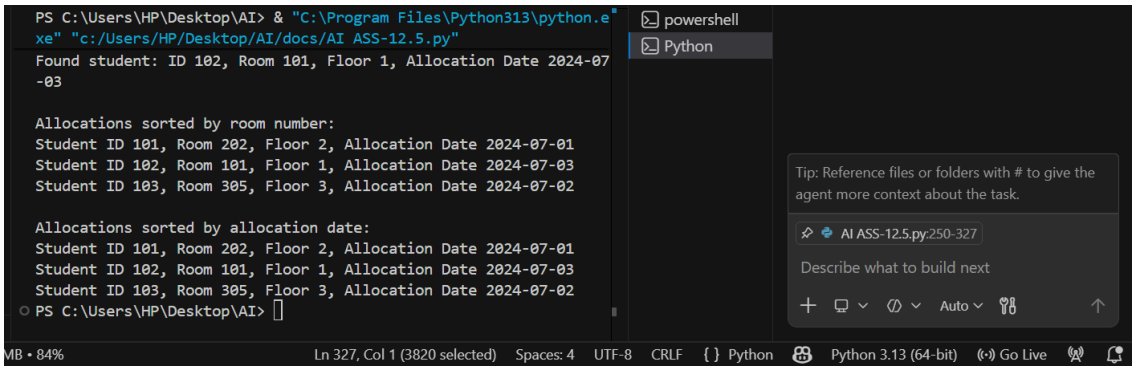
1. Search allocation details using student ID.
2. Sort records based on room number or allocation date.
   Student Task

- Use AI to suggest optimized algorithms.
- Justify the selections.
- Implement the solution in Python.

```
#Task-5: Smart Hostel Room Allocation System
#Prompt:A hostel management system stores student room allocation details inc
luding student ID, room number, floor, and allocation date. The system needs
to:
#Search allocation details using student ID.
#Sort records based on room number or allocation date.
#Suggest optimized algorithms.Justify the selections.Implement the solution i
n Python.
class RoomAllocation:
    def __init__(self, student_id, room_number, floor, allocation_date):
        self.student_id = student_id
        self.room_number = room_number
        self.floor = floor
        self.allocation_date = allocation_date
class HostelSystem:
    def __init__(self):
        self.allocations = {}
    # Add room allocation record (Using student ID as key)
    def add_allocation(self, allocation):
        self.allocations[allocation.student_id] = allocation
    # Search allocation details by student ID (Optimized search using Hashin
g)
    def search_by_student_id(self, student_id):
        return self.allocations.get(student_id, None)
    # Merge Sort for sorting by room number or allocation date
    def merge_sort(self, allocations, key):
        if len(allocations) > 1:
            mid = len(allocations) // 2
            left_half = allocations[:mid]
            right_half = allocations[mid:]
            # Recursively split the list
            self.merge_sort(left_half, key)
            self.merge_sort(right_half, key)
            i = j = k = 0
            # Merging the lists back together
            while i < len(left_half) and j < len(right_half):
                if getattr(left_half[i], key) < getattr(right_half[j], key):
                    allocations[k] = left_half[i]
                    i += 1
                else:
```

```
                    allocations[k] = right_half[j]
                    j += 1
                k += 1
            while i < len(left_half):
                allocations[k] = left_half[i]
                i += 1
                k += 1
            while j < len(right_half):
                allocations[k] = right_half[j]
                j += 1
                k += 1
    # Sort records by room number
    def sort_by_room_number(self):
        self.merge_sort(list(self.allocations.values()), 'room_number')
    # Sort records by allocation date
    def sort_by_allocation_date(self):
        self.merge_sort(list(self.allocations.values()), 'allocation_date')
# Test cases
if __name__ == "__main__":
    system = HostelSystem()
    # Adding allocation records
    system.add_allocation(RoomAllocation(101, 202, 2, datetime(2024, 7, 1)))
    system.add_allocation(RoomAllocation(102, 101, 1, datetime(2024, 7, 3)))
    system.add_allocation(RoomAllocation(103, 305, 3, datetime(2024, 7, 2)))
    # Search by student ID
    student = system.search_by_student_id(102)
    if student:
        print(f"Found student: ID {student.student_id}, Room {student.room_nu
mber}, Floor {student.floor}, Allocation Date {student.allocation_date.strfti
me('%Y-%m-%d')}")
    else:
        print("Student not found.")
    # Sort by room number
    system.sort_by_room_number()
    print("\nAllocations sorted by room number:")
    for allocation in system.allocations.values():
        print(f"Student ID {allocation.student_id}, Room {allocation.room_num
ber}, Floor {allocation.floor}, Allocation Date {allocation.allocation_date.s
trftime('%Y-%m-%d')}")
    # Sort by allocation date
    system.sort_by_allocation_date()
    print("\nAllocations sorted by allocation date:")
    for allocation in system.allocations.values():
        print(f"Student ID {allocation.student_id}, Room {allocation.room_num
ber}, Floor {allocation.floor}, Allocation Date {allocation.allocation_date.s
```

```
trftime('%Y-%m-%d')}")
```



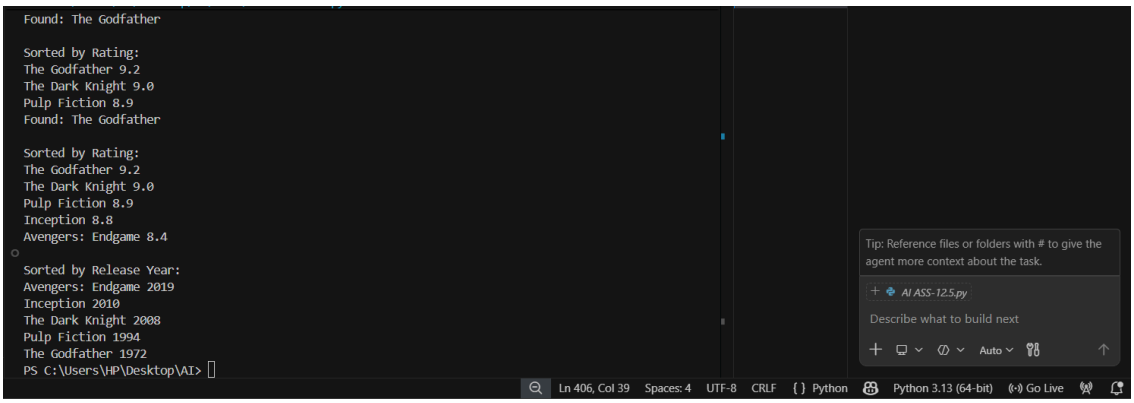
Task Description #6: Online Movie Streaming Platform
A streaming service maintains movie records with movie ID, title, genre, rating, and release year. The platform needs to:

1. Search movies by movie ID.
2. Sort movies based on rating or release year.
   Student Task
   - Recommend searching and sorting algorithms using AI.
   - Justify the chosen algorithms.
   - Implement Python functions.

```
#Task-6Create a Python program for an Online Movie Streaming Platform using O
OP.
#Store movie details (movie_id, title, genre, rating, release_year), allow se
arching by movie ID using hashing (dictionary), and sort movies by rating and
release year using Merge Sort.
#Include functions and test cases, and briefly justify the chosen algorithms.
class Movie:
    def __init__(self, movie_id, title, genre, rating, release_year):
        self.movie_id = movie_id
        self.title = title
        self.genre = genre
        self.rating = rating
        self.release_year = release_year
class MoviePlatform:
    def __init__(self):
        self.movies = {}
    # Add movie record
    def add_movie(self, movie):
```

```
        self.movies[movie.movie_id] = movie
    # Search movie by ID (Hashing - Dictionary)
    def search_by_movie_id(self, movie_id):
        return self.movies.get(movie_id, None)
    # Merge Sort
    def merge_sort(self, movies, key):
        if len(movies) > 1:
            mid = len(movies) // 2
            left_half = movies[:mid]
            right_half = movies[mid:]
            self.merge_sort(left_half, key)
            self.merge_sort(right_half, key)
            i = j = k = 0
            while i < len(left_half) and j < len(right_half):
                if getattr(left_half[i], key) > getattr(right_half[j], key):
                    movies[k] = left_half[i]
                    i += 1
                else:
                    movies[k] = right_half[j]
                    j += 1
                k += 1
            while i < len(left_half):
                movies[k] = left_half[i]
                i += 1
                k += 1
            while j < len(right_half):
                movies[k] = right_half[j]
                j += 1
                k += 1

    # Sort by rating
    def sort_by_rating(self):
        movie_list = list(self.movies.values())
        self.merge_sort(movie_list, 'rating')
        return movie_list

    # Sort by release year
    def sort_by_release_year(self):
        movie_list = list(self.movies.values())
        self.merge_sort(movie_list, 'release_year')
        return movie_list


# Test
if __name__ == "__main__":
```

```
    platform = MoviePlatform()

    platform.add_movie(Movie(1, "Inception", "Sci-Fi", 8.8, 2010))
    platform.add_movie(Movie(2, "The Dark Knight", "Action", 9.0, 2008))
    platform.add_movie(Movie(3, "The Godfather", "Crime", 9.2, 1972))
    platform.add_movie(Movie(4, "Pulp Fiction", "Crime", 8.9, 1994))
    platform.add_movie(Movie(5, "Avengers: Endgame", "Action", 8.4, 2019))

    movie = platform.search_by_movie_id(3)
    if movie:
        print("Found:", movie.title)

    print("\nSorted by Rating:")
    for m in platform.sort_by_rating():
        print(m.title, m.rating)

    print("\nSorted by Release Year:")
    for m in platform.sort_by_release_year():
        print(m.title, m.release_year)
```



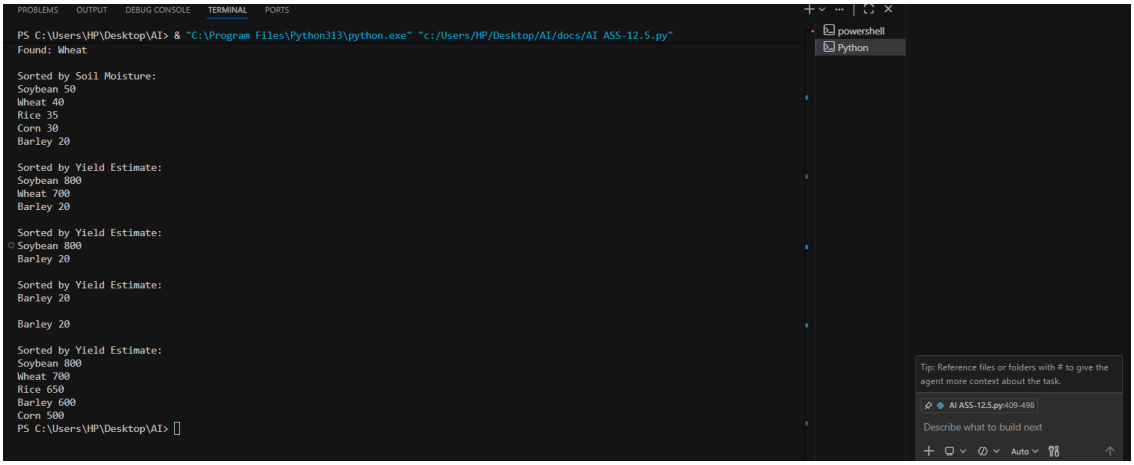
Task Description #7: Smart Agriculture Crop Monitoring System
An agriculture monitoring system stores crop data with crop ID, crop name, soil moisture level, temperature, and yield estimate. Farmers need to:

1. Search crop details using crop ID.
2. Sort crops based on moisture level or yield estimate.
   Student Task
   - Use AI-assisted reasoning to select algorithms.
   - Justify algorithm suitability.
   - Implement searching and sorting in Python.

```
 #Task-7: Smart Agriculture Crop Monitoring System
#Prompt:An agriculture monitoring system stores crop data with crop ID, crop
name, soil moisture level, temperature, and yield estimate. Farmers need to:
#Search crop details using crop ID.
#Sort crops based on moisture level or yield estimate.
#Justify algorithm suitability.Implement searching and sorting in Python.
class Crop:
    def __init__(self, crop_id, crop_name, soil_moisture, temperature, yield_
estimate):
        self.crop_id = crop_id
        self.crop_name = crop_name
        self.soil_moisture = soil_moisture
        self.temperature = temperature
        self.yield_estimate = yield_estimate


class AgricultureSystem:
    def __init__(self):
        self.crops = {}

    # Add crop
    def add_crop(self, crop):
        self.crops[crop.crop_id] = crop

    # Search crop by ID (Hashing)
    def search_by_crop_id(self, crop_id):
        return self.crops.get(crop_id, None)

    # Merge Sort
    def merge_sort(self, crops, key):
        if len(crops) > 1:
            mid = len(crops) // 2
            left = crops[:mid]
            right = crops[mid:]

            self.merge_sort(left, key)
            self.merge_sort(right, key)

            i = j = k = 0

            while i < len(left) and j < len(right):
                if getattr(left[i], key) > getattr(right[j], key):
                    crops[k] = left[i]
                    i += 1
```

```
                else:
                    crops[k] = right[j]
                    j += 1
                k += 1

            while i < len(left):
                crops[k] = left[i]
                i += 1
                k += 1

            while j < len(right):
                crops[k] = right[j]
                j += 1
                k += 1

    # Sort by soil moisture
    def sort_by_soil_moisture(self):
        crop_list = list(self.crops.values())
        self.merge_sort(crop_list, 'soil_moisture')
        return crop_list

    # Sort by yield estimate
    def sort_by_yield_estimate(self):
        crop_list = list(self.crops.values())
        self.merge_sort(crop_list, 'yield_estimate')
        return crop_list


if __name__ == "__main__":
    system = AgricultureSystem()

    system.add_crop(Crop(101, "Corn", 30, 25, 500))
    system.add_crop(Crop(102, "Wheat", 40, 22, 700))
    system.add_crop(Crop(103, "Rice", 35, 28, 650))
    system.add_crop(Crop(104, "Soybean", 50, 20, 800))
    system.add_crop(Crop(105, "Barley", 20, 30, 600))

    crop = system.search_by_crop_id(102)
    if crop:
        print("Found:", crop.crop_name)

    print("\nSorted by Soil Moisture:")
    for c in system.sort_by_soil_moisture():
        print(c.crop_name, c.soil_moisture)
```

```
    print("\nSorted by Yield Estimate:")
    for c in system.sort_by_yield_estimate():
        print(c.crop_name, c.yield_estimate)
```



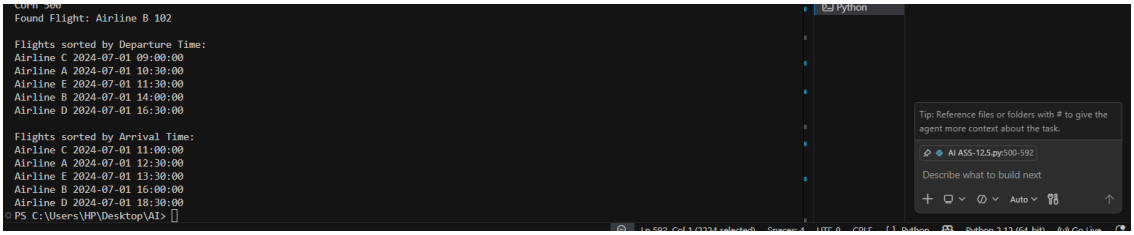
Task Description #8: Airport Flight Management System
An airport system stores flight information including flight ID, airline name, departure time, arrival time, and status. The system must:

1. Search flight details using flight ID.
2. Sort flights based on departure time or arrival time.
   Student Task
   - Use AI to recommend algorithms.
   - Justify the algorithm selection.
   - Implement searching and sorting logic in Python.

```
#Task-8: Airport Flight Management System
#Prompt:An airport system stores flight information including flight ID,airli
ne name, departure time, arrival time, and status. The systemmust:
#Search flight details using flight ID.
#Sort flights based on departure time or arrival time.
#Justify the algorithm selection.Implement searching and sorting logic in Pyt
hon.
from datetime import datetime

class Flight:
    def __init__(self, flight_id, airline_name, departure_time, arrival_time,
status):
        self.flight_id = flight_id
        self.airline_name = airline_name
```

```
        self.departure_time = departure_time
        self.arrival_time = arrival_time
        self.status = status


class AirportSystem:
    def __init__(self):
        self.flights = {}

    # Add flight
    def add_flight(self, flight):
        self.flights[flight.flight_id] = flight

    # Search flight by ID (Hashing)
    def search_by_flight_id(self, flight_id):
        return self.flights.get(flight_id, None)

    # Merge Sort
    def merge_sort(self, flights, key):
        if len(flights) > 1:
            mid = len(flights) // 2
            left = flights[:mid]
            right = flights[mid:]

            self.merge_sort(left, key)
            self.merge_sort(right, key)

            i = j = k = 0

            while i < len(left) and j < len(right):
                if getattr(left[i], key) < getattr(right[j], key):
                    flights[k] = left[i]
                    i += 1
                else:
                    flights[k] = right[j]
                    j += 1
                k += 1

            while i < len(left):
                flights[k] = left[i]
                i += 1
                k += 1

            while j < len(right):
                flights[k] = right[j]
```

```
                j += 1
                k += 1

    # Sort by departure time
    def sort_by_departure_time(self):
        flight_list = list(self.flights.values())
        self.merge_sort(flight_list, 'departure_time')
        return flight_list

    # Sort by arrival time
    def sort_by_arrival_time(self):
        flight_list = list(self.flights.values())
        self.merge_sort(flight_list, 'arrival_time')
        return flight_list


if __name__ == "__main__":
    system = AirportSystem()

    system.add_flight(Flight(101, "Airline A", datetime(2024,7,1,10,30), date
time(2024,7,1,12,30), "On Time"))
    system.add_flight(Flight(102, "Airline B", datetime(2024,7,1,14,0), datet
ime(2024,7,1,16,0), "Delayed"))
    system.add_flight(Flight(103, "Airline C", datetime(2024,7,1,9,0), dateti
me(2024,7,1,11,0), "On Time"))
    system.add_flight(Flight(104, "Airline D", datetime(2024,7,1,16,30), date
time(2024,7,1,18,30), "On Time"))
    system.add_flight(Flight(105, "Airline E", datetime(2024,7,1,11,30), date
time(2024,7,1,13,30), "On Time"))

    flight = system.search_by_flight_id(102)
    if flight:
        print("Found Flight:", flight.airline_name, flight.flight_id)

    print("\nFlights sorted by Departure Time:")
    for f in system.sort_by_departure_time():
        print(f.airline_name, f.departure_time)

    print("\nFlights sorted by Arrival Time:")
    for f in system.sort_by_arrival_time():
        print(f.airline_name, f.arrival_time)
```


Found Flight: Airline B 102
Flights sorted by Departure Time:
Airline C 2024-07-01 09:00:00
Airline A 2024-07-01 10:30:00
Airline E 2024-07-01 11:30:00
Airline B 2024-07-01 14:00:00
Airline D 2024-07-01 16:30:00
Flights sorted by Arrival Time:
Airline C 2024-07-01 11:00:00
Airline A 2024-07-01 12:30:00
Airline E 2024-07-01 13:30:00
Airline B 2024-07-01 16:00:00
Airline D 2024-07-01 18:30:00
PS C:\Users\HP\Desktop\AI>
Python
Tip: Reference files or folders with # to give the agent more context about the task.
AI ASS-12.5.py:500-592
Describe what to build next
Auto